The ecosystem of a Xen platform

dom0: The first VM that comes up on a Xen system is referred to as dom0, and it has extra privileges compared to other VMs that come up later. It provides all the services mentioned below.

XenStore: Configuration information, such as which block/network devices are associated with a domU, is written to XenStore by dom0, which is, in turn, used by domU to boot up [similar to what the BIOS provides for an OS that comes up on metal]. It also provides information about currently active domains. Dynamic reconfiguration of guest domains is made possible by XenStore. For example, you can pass additional disk devices from dom0 to domU, while domU is running.

xend: This provides the interface between the hypervisor and the rest of the user-space tools. It also provides a stable API for the higher-level management tools to be built without worrying about hypervisor changes. xend performs most of the work involved in a guest domain's lifecycle—creation, shutdown, suspension and migration, in response to admin commands from dom0. It writes the resource information associated with a guest to XenStore, besides listening to a Unix socket as well as httpd—hence making local as well as remote management of a Xen system, possible.

XenBus: A guest kernel listens on a XenBus, waiting for any events that will result in dynamic reconfiguration of the guest. XenStore delivers those events from dom0.

Pygrub: It understands the guest filesystems, and reads the kernel binary, boot archive binary and any other module required to boot up a guest system.

libvirt and tools: As more virtualisation technologies are hitting the market, high-level management tools should be immune to the underlying hypervisor. The libvirt library provides such hypervisor agnostic APIs to build higher-level management tools. virt-install (1) is one such tool, built on top of libvirt to install a guest. virsh (1), the basic administrative tool to interact with the Xen system, is also built on top of libvirt.

vdiskadm: It allows us to create virtual disk images of various formats—vmdk (VMware format), vdi (VirtualBox format), vhd (Hyper-V) and raw. You can take snapshots and make clones of vdisk images for ease of management. virt-convert (1) allows conversion of one disk format to another disk format. By default, OpenSolaris uses the vmdk format.

vdisk: For each virtual disk passed to a guest domain, there is one vdisk process running in the user land of the dom0, to coordinate the IO. This prevents proliferation of 'disk format' knowledge in the hypervisor or dom0 kernel.

libvirtd: On an OpenSolaris-based Xen system, management tools communicate with xend via libvirtd to improve security.

Log files: Detailed logs of hypervisor activities and domain life cycle events are available in the */var/log/xen/* directory.

Machinations of the Xen hypervisor

Hypercall interface: In the same way applications make use of system calls to request the services of the kernel, a guest kernel makes hypercalls to request the services of the hypervisor. Applications running on top of guest kernels continue to run unmodified.

Virtualising the CPU: x86 CPUs provide four privilege levels—0 to 3, with 0 being the highest privilege level and 3 being the lowest. Because of the multiple rings of protection, it is easy to virtualise x86-based CPUs. In 32-bit mode, Xen runs in Ring 0, the guest kernel runs in Ring 1, and applications, as usual, run in Ring 3. In 64-bit mode, Xen runs in Ring 0, while the guest kernel and applications run in Ring 3 due to lack of segmentation support.

Virtualising the MMU: As x86-based CPUs do hardware page table walk to fill in TLB miss, it is hard to virtualise MMU without losing out on performance. That is where paravirtualisation helps a lot. Here, guests have direct access to the physical pages and they continue to build and maintain the page tables. Xen steps in to ensure that page table pages don't get modified without its knowledge, so it marks them as 'read only'. Any updates to them by the guest kernel are trapped and validated before being updated.

Virtualising the IDT: The *set_trap_table()* hypercall is used by the guests to register their exception handlers. Whenever an exception happens, a stub routine in the Xen hypervisor executes. This will pass on the exception to the appropriate guest via event channels. Then the corresponding routine from the above trap table is invoked. Two major faults that happen very frequently on a system are: system calls and pagefaults. The throughput of the system is dependent on how these two faults are handled. Xen installs guest-specific handlers directly in the IDT, thus reducing one level of indirection in the fault handler. Xen validates these handlers before installing them in the IDT.

Virtualising the IO: This is another area where paravirtualisation helps in boosting the performance of the guest domains. Guest domains do know that they don't have direct control of the hardware. Xen provides a '*split driver model*', which gives close to native performance in IO. This works by providing a class-specific *frontend driver* in each of the guest domains; it talks to a '*backend driver*' available in dom0. [Xen also provides a mechanism whereby a particular domU can also host a peripheral—in which case that domU will provide the back-end driver.] The front-end and back-end drivers communicate using *event channels*.